

LEGOMem: 面向 workflow 自动化多智能体大语言模型系统的模块化程序性记忆

Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang,

Victor Rühle, Saravan Rajmohan

Microsoft; 通信作者: Dongge Han

摘要

作者提出 LEGOMem, 一种面向 workflow 自动化中多智能体大语言模型 (Large Language Model, LLM) 系统的模块化程序性记忆框架。LEGOMem 将历史任务轨迹分解为可复用的记忆单元, 并在编排器与任务智能体之间灵活分配这些单元, 以支持规划和执行。为探索多智能体系统中的记忆设计空间, 作者以 LEGOMem 为研究载体, 对多智能体程序性记忆开展系统研究, 考察记忆应放置在何处、应如何检索, 以及哪些智能体从中获益最多。OfficeBench 基准上的实验表明, 编排器记忆对于有效的任务分解与委派至关重要, 而细粒度智能体记忆则能提高执行准确率。作者发现, 即使由较小语言模型组成的团队, 也能从程序性记忆中显著获益: 通过复用先前执行轨迹来改善规划与工具使用的准确性, 它们可以缩小与更强智能体之间的性能差距。这些结果表明, LEGOMem 既是记忆增强智能体系统的实用框架, 也是研究多智能体 workflow 自动化中记忆设计的工具。

关键词: 多智能体系统; 程序性记忆; LLM 智能体; workflow

1 引言

大语言模型正日益被部署为智能体, 用于自动化复杂的多步骤 workflow Yao et al. (2023); Song et al. (2023); Wang et al. (2023); Kim et al. (2024); Wang et al. (2024a); Xie et al. (2024); Wang et al. (2025); Zhou et al. (2023); Mialon et al. (2023); Cao et al. (2024); Rana et al. (2023); Ahn et al. (2022); Wang et al. (2024b); Cheng et al. (2024)。这些智能体在文档编辑、电子邮件处理和日程安排等生产力环境中尤其有价值。为处理此类任务的多样性和组合性, 近期系统常采用多智能体 Stone and Veloso (2000); Wooldridge (2009) 设计, 让多个基于 LLM 的智能体跨角色和工具协作、专门化或委派职责 Fourney et al. (2024); Wu et al. (2024a); Dang et al. (2025); Chen et al. (2023); Zhang et al. (2024)。这一趋势体现了 AI 系统设计中更广泛的转变: 现实世界本身具有多智能体属性, 涉及异构角色和协同决策。多智能体 LLM 系统为推理、工具使用和工作流执行提供了可扩展、模块化的方法, 因此很适合日益复杂的生产力环境。

尽管已有这些进展, 当前多智能体系统在很大程度上仍是无状态且事务式的: 每项任务都从头求解, 无法复用以往经验。记忆的缺失, 尤其是程序性记忆的缺失, 限制了系统从过去经验中学习并随时间积累复杂 workflow 执行技能的能力。近期工作虽然已为单智能体 LLM 提出 Synapse Zheng et al. (2023)、Agent Workflow Memory (AWM) Wang et al. (2024c) 等记忆模块, 但这些方法

没有处理多智能体系统特有的协调与专门化挑战。

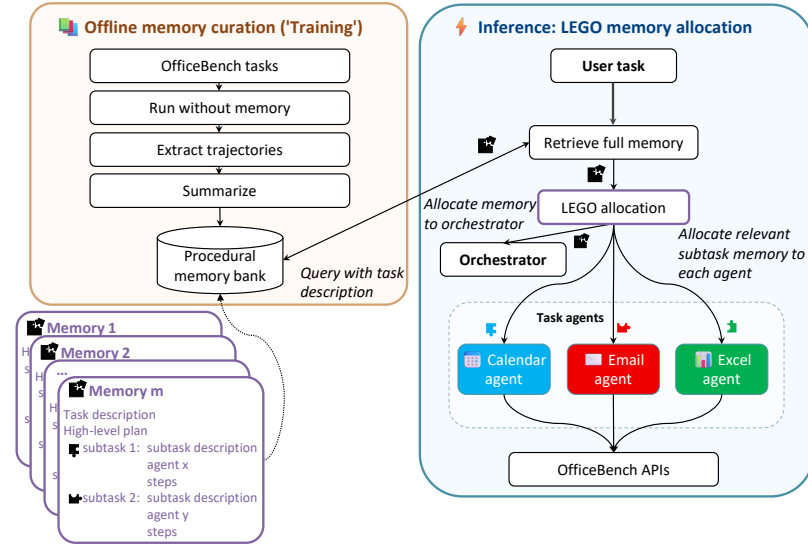
为弥补这一空缺, 作者提出专为多智能体 LLM 系统设计的模块化程序性记忆框架 LEGOMem。本文聚焦一类常见且实用的多智能体架构: 中央编排器执行规划, 并把子任务委派给使用工具的专门化任务智能体, Magentic-One 框架即为其代表 Fourney et al. (2024); Wu et al. (2024a)。作者的目标是让编排器与任务智能体都获得以历史任务轨迹为基础的记忆, 从而改善规划、协调与任务执行。为此, LEGOMem 将成功执行提炼为结构化记忆单元: 全任务记忆 (任务级规划与推理轨迹) 和子任务记忆 (智能体行为与工具交互)。这些模块化记忆存入记忆库, 以语义嵌入建立索引, 并在推理时复用来增强规划与执行。

LEGOMem 作为现有多智能体系统之上的检索增强生成 (Retrieval-Augmented Generation, RAG) 层实现 Lewis et al. (2020); Gao et al. (2023); Douze et al. (2024)。面对新任务时, 编排器获得相关的全任务记忆以支持任务分解和智能体选择, 每个任务智能体则获得与其受委派子任务相匹配的子任务记忆。作者探索了三种记忆检索策略: 基础检索、动态检索和查询重写, 以研究检索方式和记忆特异性如何影响多智能体性能。该框架使作者能够系统考察多智能体记忆设计中的关键问题, 包括记忆放在哪里、如何检索, 以及哪些智能体从中获益最多。

作者在 OfficeBench Wang et al. (2024a) 基准的生产力 workflow 自动化场景中评估 LEGOMem, 智能体团队分别采用纯 LLM、混合模型和小语言模型配置。在这些设置中, 所有 LEGOMem 变体相对于无记忆方法和基线方法都显著提高了任务成功率。消融研究显示, 编排器记忆对高层规划和委派至关重要; 细粒度子任务检索则会给更依赖局部执行支持的小模型智能体带来显著收益。这些发现揭示了记忆放置方式与检索策略如何塑造 workflow 环境中的多智能体协作效果。总体而言, LEGOMem 为记忆增强的多智能体 workflow 自动化提供了实用且可扩展的框架, 使智能体能够通过复用结构化程序性知识, 更有效地规划、协调和执行。作者希望本工作推动复杂生产力场景中记忆设计、持续学习和高效智能体协作的后续研究。

2 相关工作

面向 workflow 自动化的多智能体 LLM 系统。 LLM 的近期发展促成了能够规划、分解并解决复杂 workflow 的多智能体系统。Magentic-One Fourney et al. (2024) 等通用多智能体框架 Fourney et al. (2024); Wu et al. (2024a); Dang et al. (2025); Chen et al. (2023); Zhang et al. (2024) 通常采用一种共同设计模式: 主编排智能体把高层目标



(a) LEGOMEM 框架概览



(b) LEGOMEM 记忆结构示例

图 1: LEGOMEM 框架概览与记忆示例。多智能体系统由一个编排器和若干任务智能体组成。编排器负责规划、选择下一智能体和分配子任务；任务智能体则通过 API 工具调用与环境交互以执行子任务。为清晰起见，图中省略了 Word 智能体等其他任务智能体。

分解为逐步计划，并指挥专门化智能体团队执行具体子任务。这种模块化多智能体架构简化了开发过程，并便于复用封装后的技能，相比单体式单智能体方法具有显著优势。然而，一个关键限制依然存在：这些系统常常是无状态的，每项任务都从头求解，并丢弃执行过程中得到的有价值认识。没有记忆，智能体可能反复犯下相同错误，也无法随时间改进。

LLM 智能体的记忆。 记忆为无状态智能体的局限提供了一种自然解决方案。然而，LLM 智能体记忆面临的主要挑战是：现有设计常面向单智能体系统，并且往往属于情景记忆或语义记忆，即重放对话历史中的信息 Zhong et al. (2024); Sun and Zeng (2025); Rasmussen et al. (2025); Wu et al. (2024b); Maharana et al. (2024)。例如，A-MEM Xu et al. (2025) 把交互记录为相互连接的笔记网络并形成不断演化的记忆结构，Mem0 Chhikara et al. (2025) 则聚焦管理持续对话中的记忆。尽管这些系统推进了记忆能力，它们并非为智能体式学习和工作流自动化而设计。另一类工作 Lee et al. (2025); Kang et al. (2025); Zhou et al. (2025) 以智能体工作流的记忆优化为目标，聚焦工作流智能体的短期上下文优化。与本文最相关的长期程序性记忆研究包括 Synapse Zheng et al. (2023) 和 AWM Wang et al. (2024c)：前者把历史成功的完整轨迹用作示例，后者归纳常用子任务序列并将其作为可复用技能。然而，两者都只针对单智能体场景。相比之下，LEGOMEM 为多智能体系统引入模块化、角色感知的程序性记忆。它在编排器和任务智能体之间灵活分配记忆，从而处理记忆放置与分配方面的独特挑战，并通过更好的规划、执行和协调来改善工作流自动化。

3 LEGOMem：面向多智能体 LLM 系统的模块化程序性记忆

本节介绍为多智能体 LLM 系统设计的模块化程序性记忆框架 LEGOMEM。作者首先形式化带程序性记忆的多智能体工作流执行问题，然后给出 LEGOMEM 框架、其变体，以及实验所研究的设计选择。

3.1 问题形式化

3.1.1 面向工作流自动化的多智能体系统。

作者考虑一种常见的多智能体工作流自动化框架（基于 Magentic-One 系统 Fourney et al. (2024)）：它包含编排器 A_{orch} 、任务智能体集合 $A = \{A_1, \dots, A_k\}$ 以及外部环境 $\mathcal{E}$ 。任务 T 由自然语言描述 d 指定，并且必须在 $\mathcal{E}$ 中执行。具体而言，作者为 Word、Excel、Calendar、Email、System 和 OCR-PDF 应用实现了任务智能体。这些智能体通过工具 API 与 Docker 环境中的模拟应用交互，以保证隔离且可复现的执行。

编排器首先生成初始高层计划 $\pi_0 = \{s_1, \dots, s_m\}$ ，列出一种可能的子任务序列。不过，编排过程并非静态地遵循计划：每完成一步编排，编排器都会依据当前状态 σ_t 和智能体返回的观测动态生成下一个子任务，而不是简单地从初始计划中选择。

形式化地，在每个编排步骤 t ：

1. 编排器提出下一子任务 $s_t = \pi_{orch}(\sigma_t)$ ；
2. 将该子任务分配给适当的任务智能体 A_j ；
3. 任务智能体通过向环境 $\mathcal{E}$ 发出工具使用命令来执行 s_t ，返回观测 o_t 和执行摘要 r_t ；
4. 编排器更新状态 $\sigma_{t+1} = f(\sigma_t, r_t)$ ，并继续编排。

如果进展停滞（例如状态重复或行为陷入循环），编排器可以执行重新规划，生成修订后的高层计划 π' ，并从更新后的状态继续编排。当最终环境状态 $\sigma_{\text{final}} \in \mathcal{E}$ 满足任务目标时，系统被视为成功。

3.1.2 多智能体程序性记忆。

上述编排循环定义了智能体如何与环境交互，但它仍然是无状态的：每个新任务 T 都从头求解，并丢弃历史执行中的知识。为克服这一限制并使智能体能够从经验中改进，作者引入多智能体程序性记忆：从成功轨迹中提炼、并在多智能体系统的不同任务间复用的模块化、角色感知记忆。与主要记录事件或文本信息的情景记忆和语义记忆不同，多智能体程序性记忆把工作流抽象为同时面向编排器和任务智能体的可复用子程序。这些记忆让编排器能在更多上下文支持下改善规划和智能体选择，同时为任务智能体提供执行级指导，从而更准确、高效地使用工具。

形式上，作者把记忆存储 M 定义为从历史执行中获得的模块化记忆单元集合。其中包括捕获编排计划和执行轨迹摘要的**全任务记忆**，以及捕获智能体特定子任务执行过程的**子任务记忆**。它们共同构成角色感知记忆库，可在推理过程中检索并分配，以同时增强规划与执行。下节介绍 LEGOMem 如何通过结构化记忆构建、推理时分配，以及不同的检索和记忆复用策略来实现这一形式化定义，从而得到更鲁棒的工作流自动化。

3.2 LEGOMem 框架

LEGOMem 通过为多智能体系统配备模块化程序性记忆来实现上述问题设定。它分两个阶段运行：(i) 离线记忆构建阶段，把成功任务轨迹提炼为可复用记忆单元；(ii) 在线记忆增强推理阶段，把检索到的记忆分配给编排器和任务智能体，以指导规划与执行。如图 1a 所示，历史任务轨迹经整理后进入程序性记忆库；推理时查询该记忆库，为系统提供高层编排指导和面向特定智能体的执行轨迹。图 1b 进一步展示了这些记忆单元的结构，包括高层计划、局部智能体子任务轨迹、最终答案和简短反思。这种模块化设计使系统能够像积木一样重组历史经验，支持在多种多智能体环境中高效、可靠地完成任务。

3.2.1 记忆构建。

LEGOMem 的第一阶段是离线记忆构建，即把成功任务轨迹提炼为结构化、可复用的记忆单元。作者从每条轨迹中提取两类互补记忆：(i) 全任务记忆，记录任务描述和已执行的高层计划；(ii) 子任务记忆，封装子任务描述、局部智能体行为、工具使用和观测。这些模块化单元存入程序性记忆库 M ，供未来复用。推理时，编排器获得完整的全任务记忆，任务智能体则获得相关子任务记忆。

具体而言，构建过程处理成功完成任务的执行日志。每份日志记录任务描述、编排器的规划与编排步骤、委派给智能体的子任务，以及对应的智能体执行内容（工具使用命令、观测和结果）。作者使用一个 LLM 将这些日志转换为图 1b 所示的结构化 LEGOMem 单元。所得记忆库 M 以向量数据库实现，并使用稠密嵌入建立索引。令 $\phi(\cdot)$ 表示索引所用的嵌入模型；对于全任务记忆，系统基于任务描述 d 计算 $\phi(d)$ ，以实现语义相似度检索。

算法 1 使用基础 LEGOMem 的多智能体执行

```

1: 输入：任务描述  $d_{\text{new}}$ ，记忆库  $M$ ，编排器  $A_{\text{orch}}$ ，任务智能体  $A = \{A_1, \dots, A_k\}$ 
2: 计算  $\phi(d_{\text{new}})$ ，从  $M$  中检索语义最相似的 top- $K$  个全任务记忆  $m = \{m_1, \dots, m_K\}$ 。
3: 从全任务记忆中提取子任务记忆  $\{m_1^1, \dots, m_n^K\}$ ，并把相应子任务记忆分配给每个智能体。
4: 初始化环境  $\mathcal{E}$ ，开始任务  $d_{\text{new}}$ 。
5: 用检索到的全任务记忆  $m$  增强编排器，编排器生成初始计划  $\pi_o$ 。
6: while 任务尚未完成 do
7:   编排器  $A_{\text{orch}}$  选择下一智能体  $A_t \in A$ ，生成下一子任务  $s_t$  并分配给  $A_t$ 。
8:   用子任务记忆增强任务智能体  $A_t$ 。
9:   任务智能体  $A_t$  生成工具使用动作列表，并在环境中执行。
10:  智能体接收观测  $o_t$ ，总结子任务执行情况，并向编排器  $A_{\text{orch}}$  发送摘要消息  $r_t$ 。
11:   if 进展停滞 then
12:     编排器重新规划并更新计划  $\pi'$ 。
13:   end if
14: end while
15: 返回：编排器的最终响应。

```

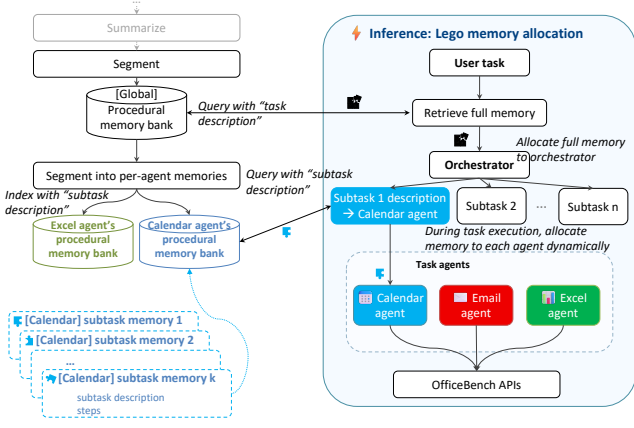
作者实现并比较三种检索与分配策略：基础 LEGOMem、LEGOMem-DYNAMIC 和 LEGOMem-QUERYREWRITE。基础 LEGOMem 使用 $\phi(d)$ 为整个记忆库 M 建立索引，因此可以在推理时直接检索相关全任务记忆，并把子任务记忆分配给任务智能体。其他两个变体则把全任务记忆的全局记忆库 M 与每个任务智能体的子任务记忆库 $\{M_{A_j} \mid A_j \in A\}$ 分离；后者包含可从全局记忆中直接提取的子任务记忆，并用子任务描述的嵌入 $\phi(d_{\text{subtask}})$ 建立索引。subsection 3.3 将进一步介绍这些变体。

3.2.2 记忆增强推理。

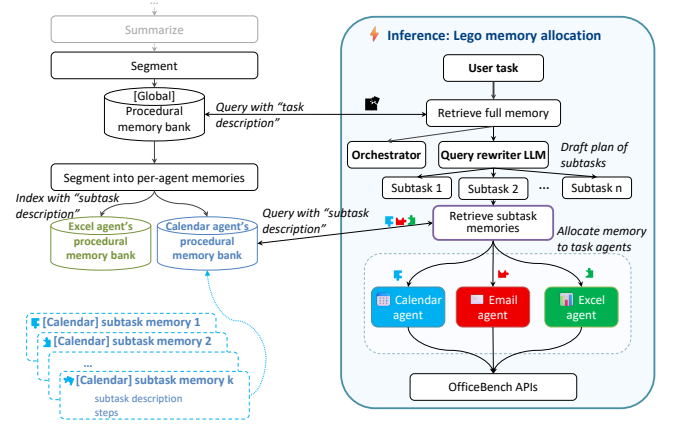
第二阶段中，LEGOMem 通过向编排器提供全任务记忆（用于端到端规划和细粒度编排），并向任务智能体提供子任务记忆（用于局部执行指导）来增强任务执行循环。给定新任务 d_{new} ，系统从记忆库中检索相关记忆并据此分配。作者设计并测试了三种采用不同记忆检索策略的 LEGOMem 变体，详见 subsection 3.3。此处介绍算法 1 所示的基础 LEGOMem 推理方法。

给定描述为 d_{new} 的新任务，系统取得嵌入 $\phi(d_{\text{new}})$ ，首先依据语义相似度从全局记忆库 M 中检索 top- K 个相关记忆。随后，系统把全任务记忆分配给编排器，从检索到的全任务记忆中提取子任务记忆，并将其分配给相应任务智能体。如图 1a 所示，在基础 LEGOMem 中，编排器获得提供端到端工作流的全任务记忆，任务智能体则获得提供局部执行指导的子任务记忆。该设计使编排器能够利用历史轨迹进行有依据的规划、智能体能力定位和错误恢复，同时提高任务智能体使用工具的准确性与效率。

任务开始时，编排器接收全任务记忆并执行初始规划。此后，在每个编排步骤中，编排器同时依据当前状态和检索到的全任务记忆动态生成下一个子任务。被选中的智能体利用分配到的子任务记忆执行该任务，并返回观测和摘要以更新编排器状态。如果进展停滞，编排



(a) LEGOMEM-DYNAMIC 变体



(b) LEGOMEM-QUERYREWRITE 变体

图 2: LEGOMEM 变体比较: (a) LEGOMEM-DYNAMIC 在执行期间动态检索子任务记忆; (b) LEGOMEM-QUERYREWRITE 使用查询重写为每个子任务检索多个候选记忆。

器可以把记忆用作额外指导来重新规划。通过这一循环，LEGOMEM 将历史经验纳入规划和协调中的决策过程，从而提高多智能体工作流的可靠性与效率。

3.3 LEGOMem 变体

为研究多智能体系统中子任务检索粒度的影响，作者比较 LEGOMEM 的三个变体：基础 LEGOMEM、LEGOMEM-DYNAMIC 和 LEGOMEM-QUERYREWRITE。它们在子任务记忆的存储、检索及向任务智能体的分配方式上有所不同。

如subsection 3.2所述，基础 LEGOMEM 保持一个全局程序性记忆库 $\mathcal{M}$ 。推理时，它使用任务描述检索全任务记忆并用于增强编排器，随后直接从这些检索结果中提取子任务记忆，并静态分配给相关任务智能体。该方法简单高效，在各种团队上都取得了较强性能。不过，如果检索到的全任务记忆与当前任务的子任务结构不同，它有时无法为某些智能体呈现相关子任务记忆；即使整体任务看似相似，子任务组成也可能不同。为此，作者实现了两个支持更细粒度子任务级检索的变体，以提高智能体级记忆的相关性。

LEGOMem-Dynamic. 如图 2a所示，LEGOMEM-DYNAMIC 在执行期间进行子任务级检索。编排器记忆的存储和检索方式与基础版本保持相同，同时系统维护从全局记忆库切分得到的、面向每个智能体的子任务记忆库。当编排器为智能体 A_t 生成子任务 s_t 时，系统计算其嵌入 $\phi(s_t)$ ，并查询该智能体的记忆库 $\mathcal{M}_{A_t}$ ，只检索最相关的历史子任务轨迹。这种即时检索为任务智能体提供更精确的执行指导，并减少无关记忆造成的噪声。

LEGOMem-QueryRewrite. LEGOMEM-DYNAMIC 在每个编排步骤执行即时检索，因此会在运行期间反复进行子任务嵌入和检索。LEGOMEM-QUERYREWRITE 借助查询重写 Ma et al. (2023); Li et al. ([n.d.]) 把这一过程移到规划阶段。如图 2b所示，在检索全任务记忆后，查询重写 LLM ψ 利用这些记忆为新任务生成草拟计划 $\pi'_{\text{draft}} = \{s'_1, s'_2, \dots, s'_n\}$ ，其中包含重写后的子任务。随后，系统通过 $\phi(s'_j)$ 嵌入每个 s'_j ，并在任务执行开始前从对应智能体的记忆库 $\mathcal{M}_{A_j}$ 中检索相关子任务记忆。该方法保留了 LEGOMEM-DYNAMIC

细粒度检索的收益，同时避免运行时重复查询，使执行更高效、编排更顺畅。

值得注意的是，作者的实验显示，在完整记忆设置下，三个变体的总体性能相近，体现出跨变体的鲁棒性。消融研究还表明，当只使用任务智能体级记忆并采用小语言模型任务智能体时，LEGOMEM-DYNAMIC 和 LEGOMEM-QUERYREWRITE 优于基础 LEGOMEM。这说明，细粒度子任务检索可能为任务智能体提供相关性更高的指导，并且在编排器支持较弱时尤其有益。总体而言，LEGOMEM 框架及其变体为多智能体 LLM 系统的程序性记忆提供了一种通用、模块化的方法，使编排器与任务智能体都能从历史任务执行中学习和复用经验。下一节将在不同智能体团队配置和记忆设置下对这些变体进行实证评估。

4 实验

作者在 OfficeBench 基准上评估 LEGOMEM，将其各变体与强基线在纯 LLM、混合以及纯 SLM 多智能体团队中进行比较。除总体性能外，作者还对记忆放置、检索策略和表示形式进行消融，以分析不同设计选择的贡献。结果显示，LEGOMEM 在各种团队配置下都能持续提高任务成功率；记忆设计，尤其是编排器记忆的放置，在实现有效多智能体协调方面发挥核心作用。

4.1 实验设置

4.1.1 数据集与指标。

作者在 OfficeBench 上评估智能体。该基准由复杂度不同的多步骤办公自动化任务组成。作者将 300 个任务划分为训练集（148 个实例，用于整理记忆）和测试集（152 个实例，用于评估）。任务覆盖三个难度级别：1 级（单应用）、2 级（双应用）和 3 级（多应用工作流）。

评估指标为**成功率**，即正确解决的任务所占百分比。任务是否成功由程序根据最终环境状态评估，包括最终输出与期望输出的精确匹配或模糊关键词匹配；例如，正确更新电子表格条目、日历事件，正确发送和接收电子邮件，以及正确回答问题。

表 1: 不同记忆变体、任务难度和多智能体团队的性能比较。表中给出各 LEGOMEM 变体与基线方法的平均成功率, 每个数据点均对三个随机种子取平均。

	LLM 团队				混合 (LLM + SLM) 团队				SLM 团队			
	1 级	2 级	3 级	总体	1 级	2 级	3 级	总体	1 级	2 级	3 级	总体
基线方法												
无记忆	49.31	58.52	33.33	45.83	45.14	48.89	16.95	35.31	36.81	34.81	7.34	24.78
Synapse	59.72	75.56	43.50	58.11	46.53	68.15	29.94	46.49	36.81	42.22	20.90	32.24
AWM	54.17	58.52	35.03	48.03	43.75	55.56	18.64	37.50	35.42	36.30	12.99	26.97
本文方法												
LEGOMEM	57.99	73.33	47.46	58.44	49.31	62.22	36.16	48.03	38.89	54.07	25.42	38.16
LEGOMEM-DYNAMIC	56.25	75.56	43.79	57.12	44.44	65.93	36.16	47.59	38.89	50.37	27.12	37.72
LEGOMEM-QUERYREWRITE	54.17	72.59	42.94	55.26	47.22	66.67	40.11	50.22	36.81	48.89	26.55	36.40

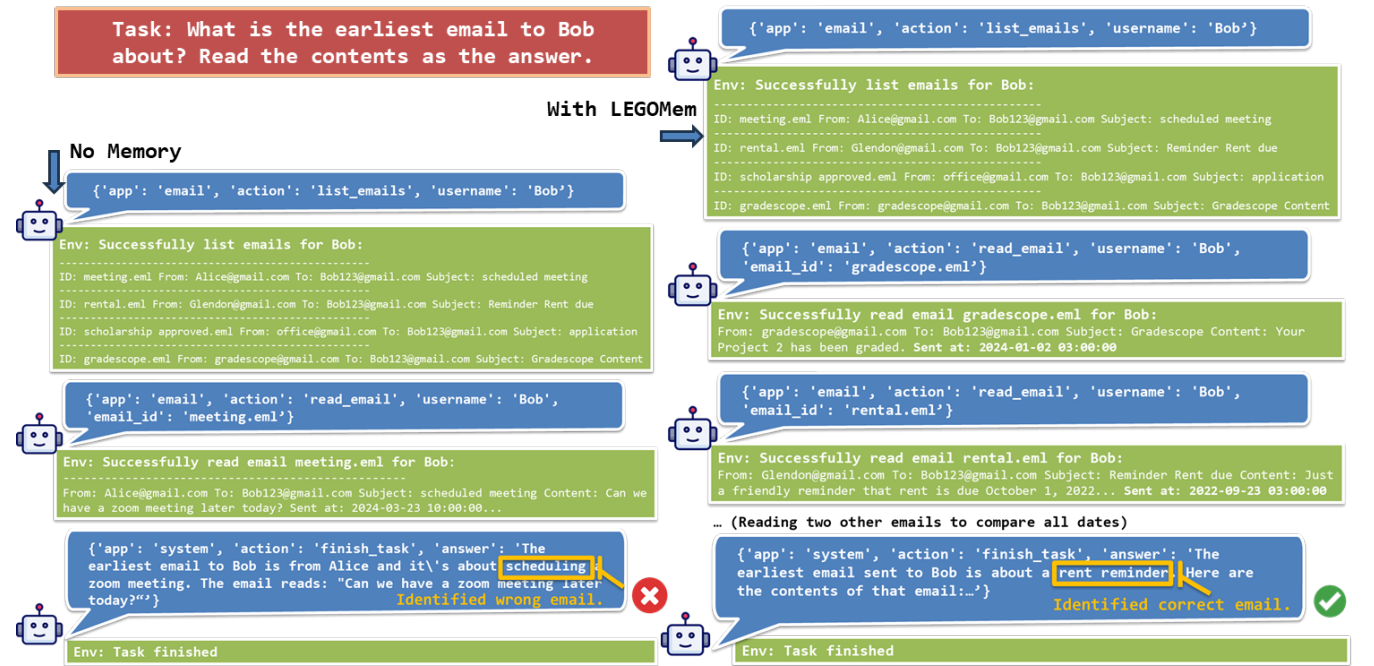


图 3: 有无记忆时智能体执行的定性示例。无记忆团队因规划不完整而未能识别最早的电子邮件, 在只读完第一封邮件后便停止; 配备 LEGOMEM 的团队则系统地读取全部邮件、取得并比较所有时间戳, 因而给出正确答案。

4.1.2 实现细节。

作者实验了三种智能体规模和能力不同的团队配置:

- **LLM 团队:** 编排器与任务智能体均使用 GPT-4o [Hurst et al. \(2024\)](#);
- **混合 (LLM + SLM) 团队:** 编排器使用 GPT-4o, 任务智能体使用 GPT-4o-mini;
- **SLM 团队:** 所有组件均使用 GPT-4o-mini [Hurst et al. \(2024\)](#)。

此外, 在记忆存储与检索中, 作者使用 OpenAI text-embedding-3-large 模型嵌入任务描述, 并使用 FAISS 库 [Douce et al. \(2024\)](#) 作为向量数据库。对于 OCR 应用, 作者使用 Phi-3.5-mini 模型 [Abdin et al. \(2024\)](#) 作为解析图像的视觉语言模型。

作者将 LEGOMEM 变体与三个基线比较: (i) **无记忆**; 以及两种面向工作流自动化程序性记忆的先进方法: (ii) **Synapse**, 使用原始动作序列和完整轨迹, 以语义相似的记忆增强智能体; (iii) **AWM**, 使用从完整轨迹中提取的子任务记忆摘要增强智能体。

4.1.3 记忆整理与智能体推理细节。

记忆构建使用 148 个训练任务。作者首先运行无记忆的完整 LLM 智能体团队, 筛选成功轨迹, 并从中提取 93 个全任务记忆。对于 LEGOMEM 变体, 作者进一步从这 93 个全任务记忆中为任务智能体提取 250 个子任务记忆。

Synapse 和 AWM 原本均面向单智能体系统。为公平比较, 作者使用相同的 93 条成功轨迹, 并把两个基线都改造用于多智能体团队, 以同时增强编排器和任务智能体。对于 Synapse, 作者用完整轨迹增强编排器和任务智能体。对于 AWM, 作者对成功轨迹聚类, 从每个簇中提取并合并子任务记忆; 推理时, 任务智能体获得相应的已提取子任务记忆, 编排器则获得已提取子任务记忆的列表。对于所有变体, 编排器使用成功轨迹中的 5 个记忆, 每个任务智能体使用 3 个记忆。

4.2 主要结果

表 1 给出主要实验结果, 比较 LEGOMEM 与基线方法在不同任务难度和智能体团队配置下的表现。

在所有场景和智能体团队配置中, LEGOMEM 变体的总体成功率均持续优于基线方法。三个 LEGOMEM 变

表 2: 不同 LEGOMem 变体下各类记忆放置机制的性能比较。

	LLM 配置				混合 (LLM + SLM) 配置			
	1 级	2 级	3 级	总体	1 级	2 级	3 级	总体
编排器 + 智能体记忆								
LEGOMem	57.99	73.33	47.46	58.44	49.31	62.22	36.16	48.03
LEGOMem-DYNAMIC	56.25	75.56	43.79	57.12	44.44	65.93	36.16	47.59
LEGOMem-QUERYREWRITE	54.17	72.59	42.94	55.26	47.22	66.67	40.11	50.22
编排器记忆 (规划) + 智能体记忆								
LEGOMem	54.86	76.30	35.03	53.51	45.14	63.70	30.51	44.96
LEGOMem-DYNAMIC	54.86	73.33	41.81	55.26	46.53	64.44	32.77	46.49
LEGOMem-QUERYREWRITE	51.39	70.37	42.94	53.73	49.31	59.26	35.59	46.93
仅编排器记忆								
LEGOMem	51.39	74.07	38.98	53.29	45.83	68.89	32.77	47.59
仅任务智能体记忆								
LEGOMem	50.00	63.70	38.98	49.78	44.44	46.67	19.21	35.31
LEGOMem-DYNAMIC	49.31	62.96	38.98	49.34	47.22	55.56	23.16	40.35
LEGOMem-QUERYREWRITE	54.86	66.67	35.03	50.66	44.44	54.81	24.29	39.69
无记忆								
无记忆	49.31	58.52	33.33	45.83	45.14	48.89	16.95	35.31

表 3: 不同 LEGOMem 变体中包含与不包含推理的记忆比较。

	LLM 团队				混合 (LLM + SLM) 团队			
	1 级	2 级	3 级	总体	1 级	2 级	3 级	总体
无推理								
LEGOMem	54.17	75.93	43.22	56.36	48.61	68.15	36.72	49.78
LEGOMem-DYNAMIC	56.94	73.33	44.63	57.02	50.00	68.15	36.72	50.22
LEGOMem-QUERYREWRITE	48.61	69.63	48.02	54.61	37.50	65.19	36.16	45.18
含推理								
LEGOMem	57.99	73.33	47.46	58.44	49.31	62.22	36.16	48.03
LEGOMem-DYNAMIC	56.25	75.56	43.79	57.12	44.44	65.93	36.16	47.59
LEGOMem-QUERYREWRITE	54.17	72.59	42.94	55.26	47.22	66.67	40.11	50.22

体表现相近且稳定，其中基础 LEGOMem 最轻量，同时取得最佳总体性能。性能提升表明，模块化的记忆表示与分配对多智能体系统有效。相较无记忆团队，LEGOMem 在 LLM、混合和 SLM 团队上的总体任务成功率分别提高 +12.61%、+12.72% 和 +13.38% 个百分点。

更重要的是，LEGOMem 使较小模型能够缩小与大模型的差距，有时甚至超过后者。例如，使用 LEGOMem-QUERYREWRITE 的混合团队达到 50.22%，超过无记忆 LLM 团队的 45.83%。同样，使用基础 LEGOMem 的纯 SLM 团队达到 38.16%，优于无记忆混合团队的 35.31%。Synapse 在 LLM 团队中仍有竞争力，反映出 LLM 解释原始程序轨迹的能力，但它在混合和 SLM 团队上的有效性不够稳定。相比之下，LEGOMem 在所有团队设置中都保持较强性能，凸显了模块化程序性记忆对于高效小模型团队的重要性。

为更直观地说明记忆对智能体行为的影响，图 3 给出一个定性案例。没有记忆时，智能体因规划不完整而未能找出最早的电子邮件，只读取第一项便停止。使用 LEGOMem 后，智能体系统地读取和比较所有邮件，正确识别最早的一封。该示例说明，LEGOMem 对推理一致性和任务完整性的改善超出了汇总成功率所能体现的范围。

4.3 消融实验

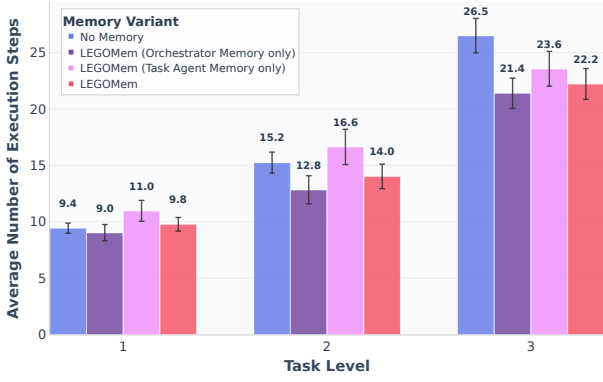
本节研究不同的记忆检索、分配和放置策略如何影响 LEGOMem 的性能。

4.3.1 记忆检索、分配与放置。

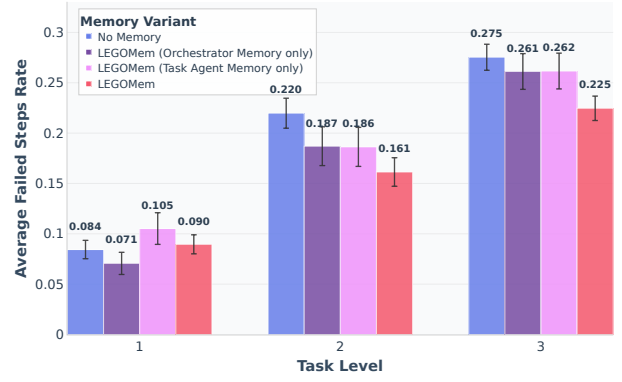
表 2 汇总了不同记忆检索变体、记忆分配策略和记忆放置设置下的消融结果。

记忆检索。 三种子任务记忆检索策略，即基础 LEGOMem、LEGOMem-DYNAMIC 和 LEGOMem-QUERYREWRITE，都表现稳健并取得相近的总体成功率。动态检索可以实现更有针对性的分配，查询重写可以提高面对子任务措辞变化时的鲁棒性，但与记忆放置和分配策略的影响相比，这些差异较小。

在仅使用任务智能体记忆的设置下，混合团队的任务智能体是较小模型，智能体级记忆的作用更关键；此时 LEGOMem-DYNAMIC 和 LEGOMem-QUERYREWRITE 的平均表现比基础 LEGOMem 高 4–5%。这些结果表明，细粒度子任务检索能够为任务智能体提供相关性和上下文性更强的指导，尤其是在全局规划信号较弱时。作者推测，在完整记忆设置下三个变体总体性能相近，可能是因为编排器记忆很强：编排器会接收历史解法的完整轨迹，并可通过更好的任务分解和委派弥补较弱任务智能体的执行能力。



(a) 各任务难度的平均执行步数



(b) 各任务难度的平均失败步骤率

图 4: 消融研究: LLM 团队采用不同 LEGOMEM 记忆放置方式时的执行步数比较。(a) LEGOMEM 变体减少了所需执行步数, 3 级任务最多下降 16.2%; (b) 步骤失败率更低, 表明程序性记忆使任务执行更可靠。

总体而言, 这些发现体现了 LEGOMEM 框架的灵活性: 即使轻量的基础变体也具有竞争力, 而在需要更细粒度记忆检索的场景中, 高级变体还能带来额外收益。

记忆分配。 对于记忆分配, 作者发现联合分配编排器记忆和任务智能体记忆 (“编排器 + 智能体记忆”变体) 取得最强的总体结果: 编排器记忆支持有效规划、任务分解和子任务编排, 任务智能体记忆则带来执行级精确性。编排器记忆具有不可或缺的作用; 当从编排器中移除记忆、只向任务智能体提供记忆时, 性能明显下降。

记忆放置。 从记忆放置看, 即使将编排器记忆限制在规划和重新规划阶段, 它仍然优于只使用任务智能体记忆的变体, 证实编排器记忆在高层规划和任务分解中的核心作用。最后, 只使用任务智能体记忆虽然有助于更准确地使用工具, 并且优于无记忆基线, 却仍不如编排器级记忆有效。这说明, 缺少全局协调时, 仅有局部记忆并不足够。

4.3.2 在记忆中加入推理的有效性。

作者还考察了使用轻量推理增强程序性记忆是否能提高性能。如表 3 所示, 差异很小: 各变体和团队类型的总体分数变化均小于两个点。例如, 基础 LEGOMEM 在 LLM 团队上略有提高 (56.36% → 58.44%), 但在混合团队上有所下降 (49.78% → 48.03%)。这些结果说明 LEGOMEM 具有鲁棒性, 其模块化结构已经能够在不增加额外推理步骤的情况下提供充分的程序性指导。

4.3.3 记忆对执行步数和失败率的影响。

作为另一项消融, 图 4 比较了 LLM 团队中, 智能体采用不同记忆放置变体时每个任务的平均执行步数, 以及因发出错误工具使用动作而产生的步骤失败率。如图 4a 所示, 与无记忆变体相比, 配备 LEGOMEM 的智能体可以减少完成任务所需的执行步数; 例如, 3 级任务从平均 26.5 步降至 22.2 步, 下降 16.2%。移除编排器记忆的“仅任务记忆”变体完成任务所需的步数多于包含编排器记忆的变体, 这是因为编排器记忆能改善规划。

类似地, 图 4b 显示 LEGOMEM 降低了智能体步骤的平均失败率。在 3 级任务上, 失败率从无记忆设置的 0.275 降至使用 LEGOMEM 时的 0.225。这些结果表明,

LEGOMEM 不仅提高任务成功率, 也使任务执行更高效、更可靠。

总之, 作者的实验显示, LEGOMEM 持续优于基线方法, 相较无记忆团队将任务成功率提高超过 12 个百分点。LEGOMEM 可以使小模型团队和混合团队达到甚至超过纯 LLM 团队, 体现其对高效多智能体配置的价值。消融还显示, 正如预期, 记忆放置策略至关重要: 编排器记忆是有效规划的关键, 而子任务记忆补充执行支持。其他分析还表明, LEGOMEM 能减少所需执行步数和逐步失败率。

5 结论

作者提出 LEGOMEM, 一种面向多智能体系统的模块化程序性记忆框架, 使编排器与任务智能体能够从历史任务执行中学习。通过把工作流表示为可复用的记忆单元, 并拆分为全任务和子任务组件, LEGOMEM 借助记忆检索与分配来支持高效的任务规划和执行。

作者实现并评估三个 LEGOMEM 变体, 以探索记忆检索和放置策略的设计空间。在工作流自动化任务上的大量实验表明, LEGOMEM 相比无记忆方法和基线方法显著提高任务成功率; 编排器记忆在规划与协调中发挥关键作用, 同时记忆也能使较小智能体受益, 体现出该框架的灵活性与有效性。

本文结果表明, 将程序性记忆整合进多智能体系统能够得到更可靠、更可复用的解法。未来工作可以探索如何也从历史失败轨迹中持续学习, 以及如何将 LEGOMEM 扩展到开放式环境和工具生态。

参考文献

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. 2024. Phi-4 technical report. *arXiv preprint arXiv:2412.08905* (2024).

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as i can, not as i say: Ground-

- ing language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).
- Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Wenjing Hu, Yuchen Mao, et al. 2024. Spider2-v: How far are multimodal agents from automating data science and engineering workflows? *Advances in Neural Information Processing Systems* 37 (2024), 107703–107744.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. 2023. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848* 2, 4 (2023), 6.
- Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, Zekai Wang, Feng Yin, Junhua Zhao, et al. 2024. Exploring large language model based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428* (2024).
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. <https://doi.org/10.48550/arXiv.2504.19413> arXiv:2504.19413 [cs].
- Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, et al. 2025. Multi-Agent Collaboration via Evolving Orchestration. *arXiv preprint arXiv:2505.19591* (2025).
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. 2024. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468* (2024).
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2025. ACON: Optimizing Context Compression for Long-horizon LLM Agents. *arXiv preprint arXiv:2510.00615* (2025).
- Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Ghohami. 2024. An llm compiler for parallel function calling. In *Forty-first International Conference on Machine Learning*.
- Dongjun Lee, Juyong Lee, Kyuyoung Kim, Jihoon Tack, Jinwoo Shin, Yee Whye Teh, and Kimin Lee. 2025. Learning to contextualize web pages for enhanced decision making by LLM agents. *arXiv preprint arXiv:2503.10689* (2025).
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- Zhicong Li, Jiahao Wang, Hangyu Mao, ZhiShu Jiang, Zhongxia Chen, Du Jiazhen, Fuzheng Zhang, Di ZHANG, and Yong Liu. [n.d.]. DMQR-RAG: Diverse Multi-Query Rewriting in Retrieval-Augmented Generation. ([n. d.]).
- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5303–5315.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753* (2024).
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*.
- Krishan Rana, Jesse Haviland, Sourav Garg, Jad AbouChakra, Ian Reid, and Niko Suenderhauf. 2023. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv preprint arXiv:2307.06135* (2023).
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956* (2025).
- Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*. 2998–3009.

- Peter Stone and Manuela Veloso. 2000. Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots* 8, 3 (2000), 345–383.
- Haoran Sun and Shaoning Zeng. 2025. Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents. *arXiv preprint arXiv:2507.22925* (2025).
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024b. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *arXiv preprint arXiv:2305.04091* (2023).
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025. OdysseyBench: Evaluating LLM Agents on Long-Horizon Complex Office Application Workflows. *arXiv preprint arXiv:2508.09124* (2025).
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024a. Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056* (2024).
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024c. Agent Workflow Memory. <https://doi.org/10.48550/arXiv.2409.07429> arXiv:2409.07429 [cs].
- Michael Wooldridge. 2009. *An Introduction to MultiAgent Systems* (2nd ed.). Wiley Publishing.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024b. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813* (2024).
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024a. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.
- Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. <https://doi.org/10.48550/arXiv.2502.12110> arXiv:2502.12110 [cs].
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. 2024. Ufo: A ui-focused agent for windows os interaction. *arXiv preprint arXiv:2402.07939* (2024).
- Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2023. Synapse: Trajectory-as-exemplar prompting with memory for computer control. *arXiv preprint arXiv:2306.07863* (2023).
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).
- Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. *arXiv preprint arXiv:2506.15841* (2025).